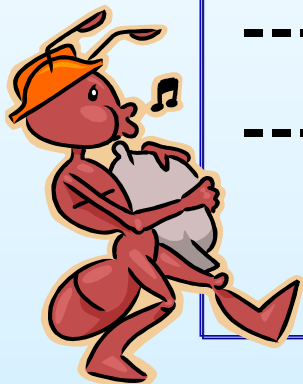


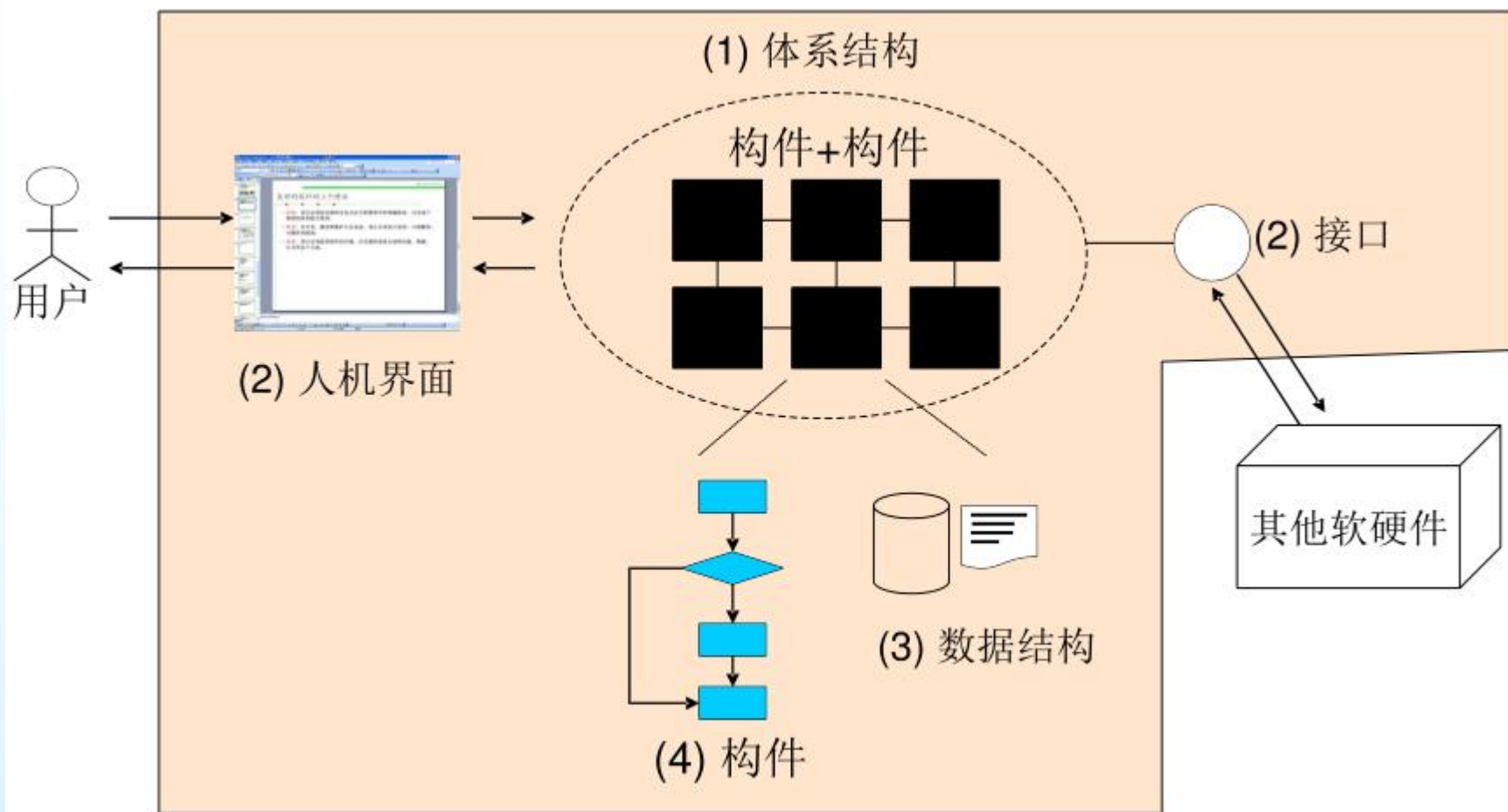
第4章：系统概要设计— 2.体系结构设计

• 学习目的和要求

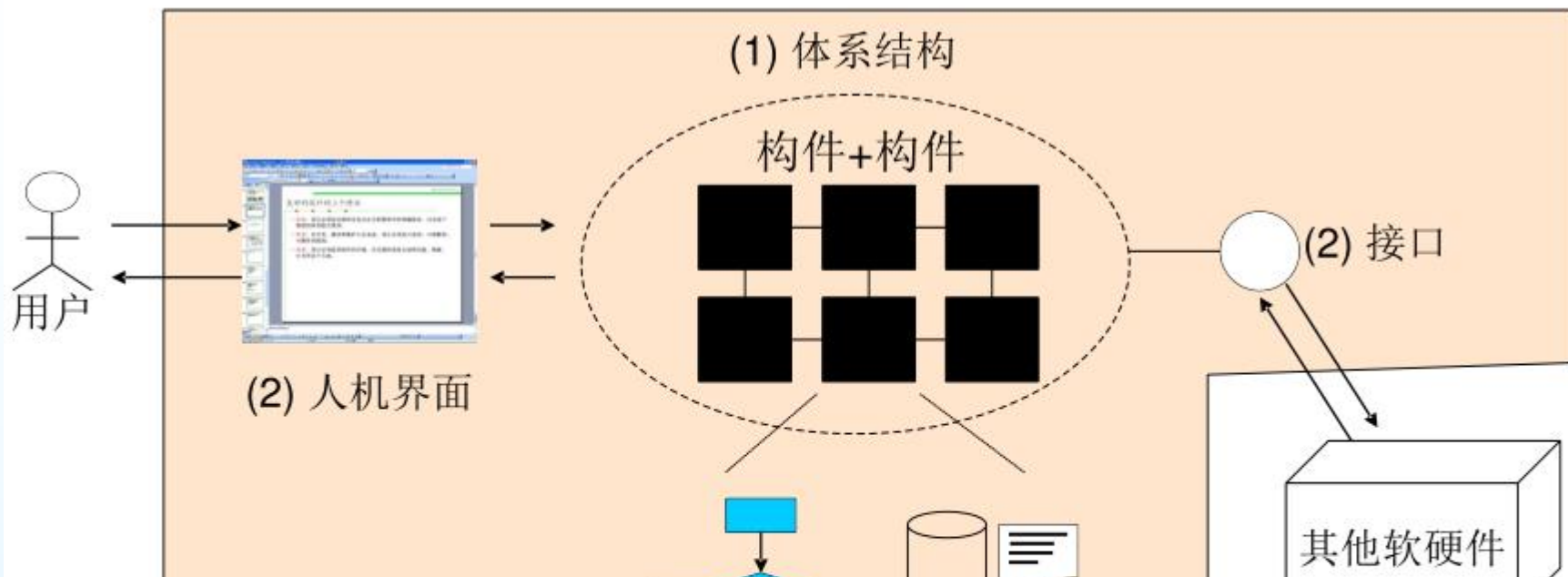
- 软件体系结构的概念—**掌握**
- 常用的体系结构风格—**理解**



□ 概要设计引言



□ 概要设计引言



- 软件系统将采用什么样的体系结构；
- 需要创建哪些功能模块以及模块间的关系；
- 公共的数据结构；
- 需要什么样的而网络环境支持；
- 需要什么类型的后台数据库。

一、软件体系结构的概念

1、软件体系结构的提出

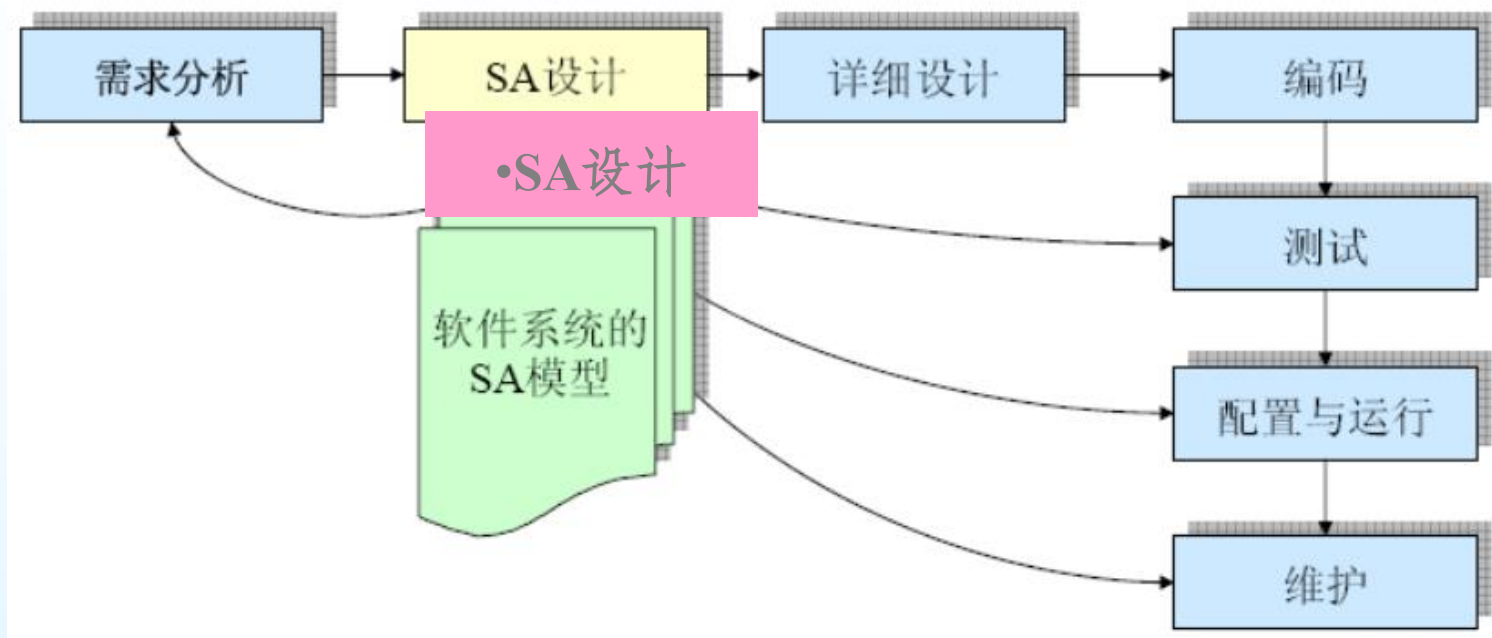
自从软件系统首次被分成许多模块，模块之间有相互作用，组合起来有整体的属性，就具有了体系结构。起初，人们把软件设计的重点放在数据结构和算法的选择上，随着软件系统规模和复杂度不断增大，系统的结构和规格说明显得越来越重要。



- ◆ 最早指出SA的重要性的是大师艾兹格·迪科斯彻（荷兰Leiden大学计算机科学家，1972年图灵奖获得者）
Edsger Wybe Dijkstra (1930-2002)
“..the larger the project, the more essential the structuring!”(1968)

一、软件体系结构的概念

1、软件体系结构的提出



■ 软件体系结构在软件生命周期中的位置

- SA在系统开发的全过程中起中心作用
- SA是设计/开发的起点和依据
- SA是配置、运行和维护的指南

一、软件体系结构的概念

2、软件体系结构的定义

软件体系结构是一个复杂软件的高层结构，它高度抽象，超越了算法和数据结构，两个基本着眼点是系统的结构及需求与实现间的交互。

定义 1： IEEE 610.12—1990软件工程标准词汇中的定义：体系结构是以构件、构件之间的关系、构件与环境之间的关系为内容的某一系统的基本组织结构，及指导上述内容设计与演化的原理。

Architecture={component, connector, environment, principle}.

定义 2： 卡耐基梅隆大学(CMU)计算机学院教授DavidGarlan和DewnePerry在IEEE软件工程学报上又采用如下的定义：软件体系结构是一个程序/系统各构件的结构、它们之间的相互关系以及进行设计的原则和随时间进化的指导方针。

SA={components, connectors, constrains}.

一、软件体系结构的概念

2、软件体系结构的定义

◎ 对定义的理解

根据以上的定义我们可以看出，从不同的角度出发，研究者对软件体系结构的定义不同，但具有以下一些共同的特性：

- ① 是一个关于系统形式和结构的综合框架，包括构件和构件的整合
- ② 这些要素应该满足一定的限制，遵循一定的设计规则，能够在一定的环境下进行演化
- ③ 反应系统开发中具有重要影响的设计决策，便于各种人员的交流，反应多种关注，据此开发的系统能完成系统既定的功能和性能

体系结构=构件+连接件+约束+质量

一、软件体系结构的概念

3、体系结构的核心元素

(1) 构件 (Component)

◎ **构件的定义：**构件是指语义完整、语法正确和有可重用价值的单位软件，是软件重用过程中可以明确辨识的系统；结构上，它是语义描述、通讯接口和实现代码的复合体。

◎ 构件的几个基本属性

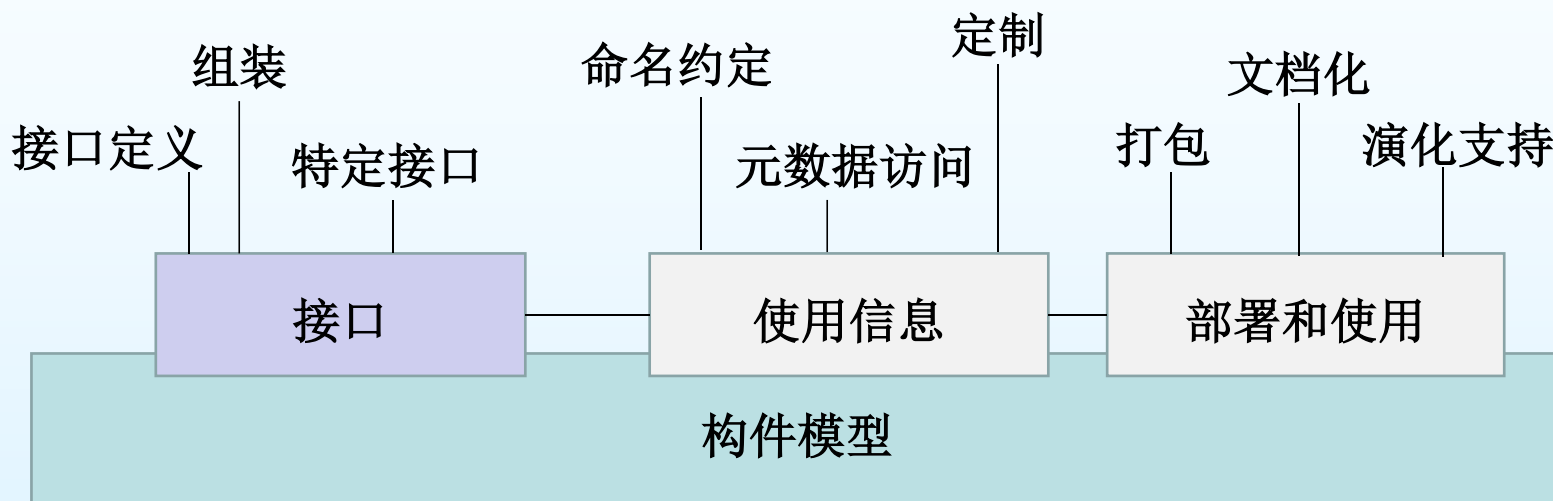
- 构件是可独立配置的单元，因此构件必须自包容；
- 构件强调与环境和其他构件的分离，因此构件的实现是严格封装的，外界没有机会或没有必要知道构件内部的实现细节；
- 构件可以在适当的环境中复合使用，因此构件需要提供清楚的接口规范，可以与环境交互。

一、软件体系结构的概念

3、体系结构的核心元素

(1) 构件模型

理想构件模型的基本要素：构件实现、文档化及开发的标准。



为使构件远程分布和访问，需要给构件一个全局唯一的名字或句柄。而且必须有一个规格说明，指出如何打包构件使其部署为一个独立的可执行体。

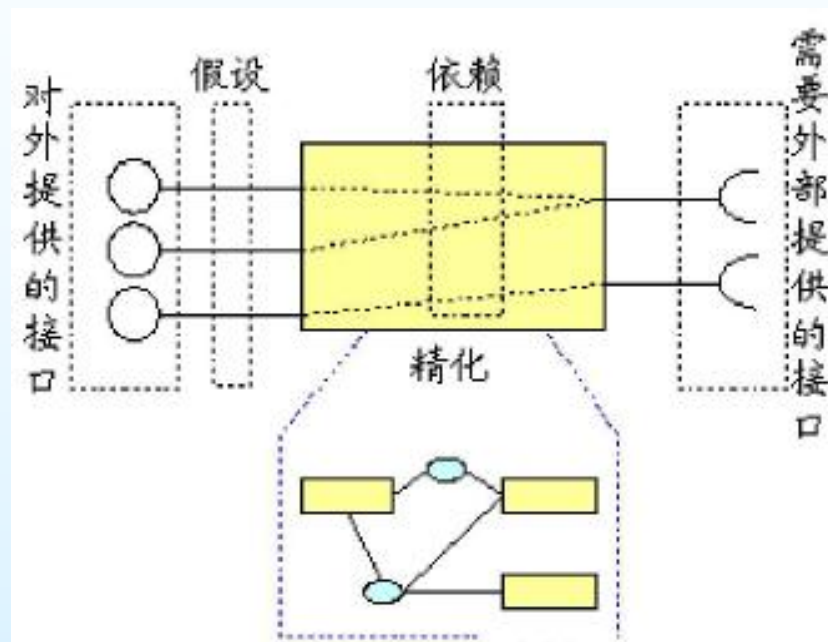
一、软件体系结构的概念

3、体系结构的核心元素

(1) 构件模型

构件对外提供的接口和需要外部提供的接口。

这两个接口的规约使得构件成为一个相对独立的软件实体。
构件使用者仅仅通过这两类接口的规约来使用构件。这两类接口中，对外提供的接口对构件模型而言是必须的，而构件是否需要外部提供的接口则取决于构件的实现方式。



接口定义中应包括操作名、参数及异常等。

一、软件体系结构的概念

3、体系结构的核心元素

(1) 构件 (Component)

- 纯计算单元构件：这类构件只有简单的输入/输出处理关联，对它们的处理一般也不保留处理状态，如数学函数、过滤器和转换器；
- 数据存储单元构件：具有永久存储特性的结构化数据，如数据库、文件系统、符号表和超文本等。
- 管理器构件：对系统中有关状态和紧密相关操作进行规定与限制的实体，如抽象数据类型和系统服务器等。
- 控制器构件：控制和管理系统中有关事件发生的时间序列，如调度程序和同步处理协调程序等。

一、软件体系结构的概念

3、体系结构的核心元素

(2) 连接件 (Connector)

连接件：用来建立构件间的交互以及支配这些交互规则的体系结构构造模块。

● 连接件接口由一组角色组成，每个角色定义参与交互的参与者

● 可以通过不同的实现系统的编译单元实现，如管道、过程调用、事件广播、C/S通信、数据库的SQL连接实现。

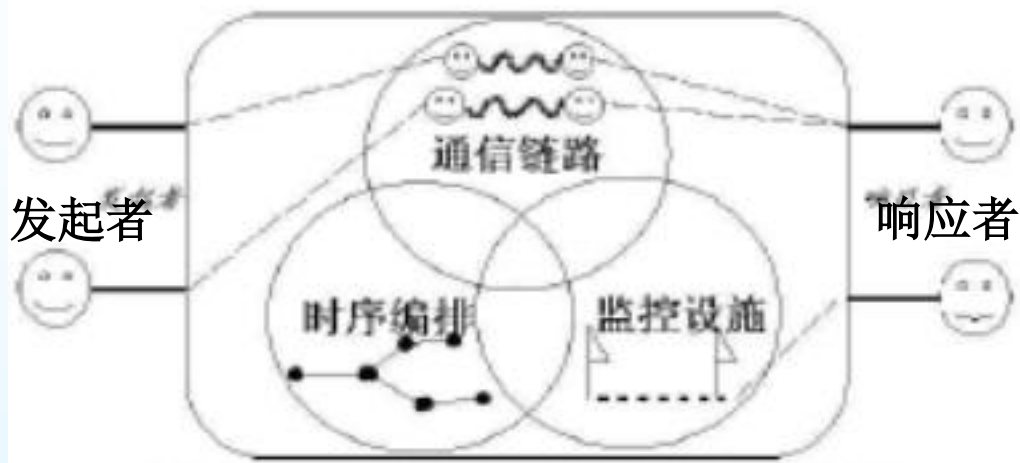
■ **连接子与构件的区别：**构件和连接子是网构软件体系结构的基本组成元素，其中构件主要负责进行业务层面的计算，连接子主要对构件之间的交互进行抽象。

一、软件体系结构的概念

3、体系结构的核心元素

(2) 连接件 (Connector)

连接子的内部规约定义了连接子对构件交互的处理细节，主要包括通信链路、时序编排和监控设施三部分。



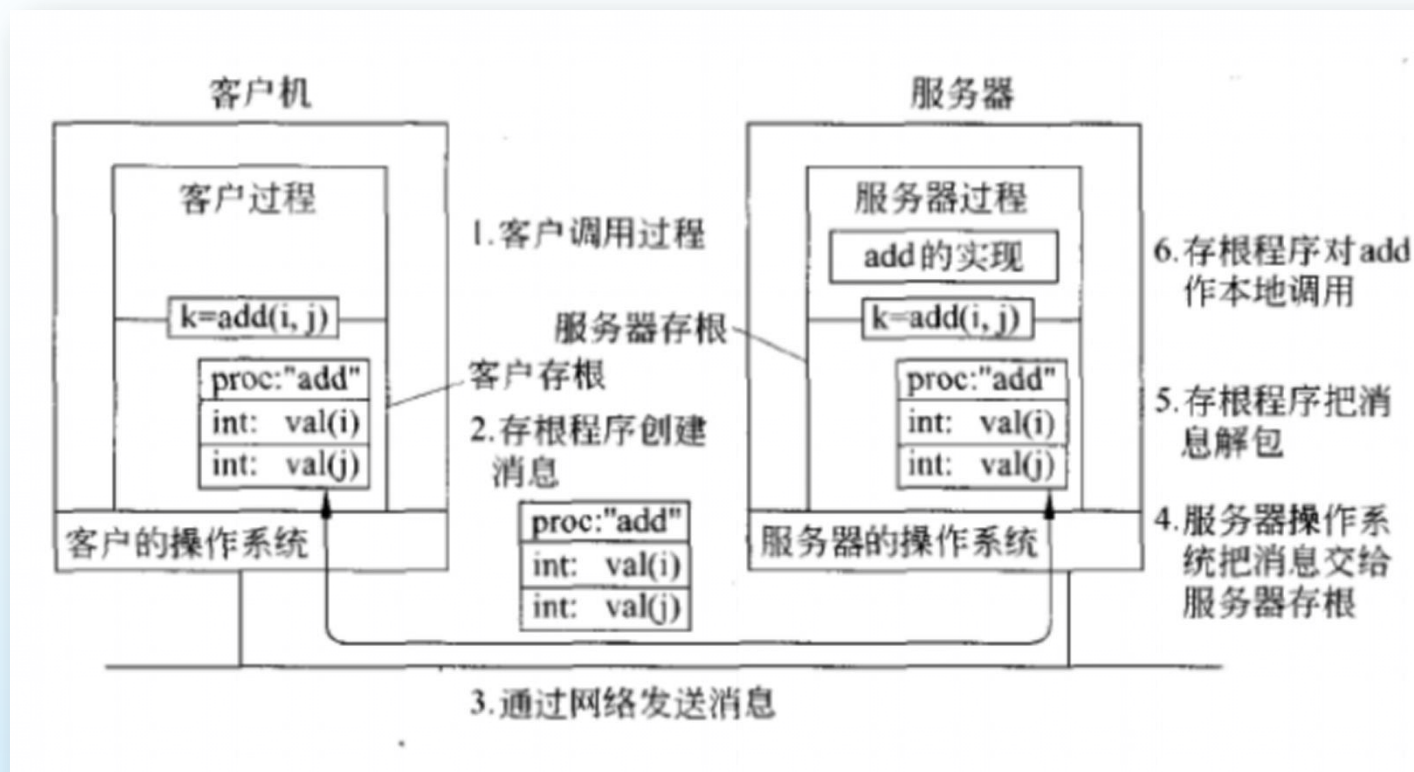
- ★ 所谓的“通信链路”是对进行通信的构件对的抽象，常见的通信协议包括过程调用、远程过程调用、消息传递等。
- ★ 对于复杂的构件交互，网构连接子模型还允许设计人员定义构件之间的交互时序，即多个构件交互顺序、并发等信息。
- ★ 连接子可对构件交互进行监控，在交互前或者交互后进行特定应用的处理。

一、软件体系结构的概念

3、体系结构的核心元素

(2) 连接件 (Connector)

过程调用连接器：在构件实体之间实现单线程控制的连接机制，如普通过程调用和远程过程调用等。



通过参数值传递的远程过程调用

一、软件体系结构的概念

3、体系结构的核心元素

(2) 连接件 (Connector)

- **数据流连接件**：系统中通过数据流进行交互的独立处理流程连接机制，其最显著的特点是根据得到的数据来进行构件实体的交互控制。如串行批处理，进程间传递的是临时文件；**Unix** 操作系统中的管道机制传递的是**Stream**；传统的编译器，一个阶段（包括词法分析、语法分析、语义分析和代码生成）的输出是另一个阶段的输入。
- **隐含触发器连接件**：由并发出现的事件来实现构件实体之间交互的连接机制，在这种连接机制中，构件实体之间不存在明确确定的交互规定，如事件调度和自动垃圾回收等。
- **消息传递类连接件**：独立构件实体之间通过离散和非在线的数据进行交互的连接机制，如**TCP/IP**等。

一、软件体系结构的概念

3、体系结构的核心元素

(3) 规约（协议Protocol）

- 连接的协议(Protocol)是连接的规约(Specification);
- 连接的规约是建立在物理层之上的有意义信息形式的表达规定;
- **目的：**使双方能够相互理解对方所发来的信息的语义。

—对过程调用来说：参数的个数和类型、参数排列次序；

例： `double GetHighestScore(int CourseID,String classID) { ...}`

—对消息传送来说：消息的格式

例： `class Message { int msgNo;String bookName;String status;}`

二、软件体系结构风格（模式）

1、体系结构风格的概念

□ 软件体系结构设计的内容：

- 选择适合于需求的软件体系结构风格，也称为软件架构软件架构设计；
- 如何以最佳的方式划分一个系统，如何标识构件，构件之间如何通信，系统元素如何能够独立地进化。



- 在设计软件体系结构时可以借鉴或采用某种特定的软件按体系结构风格。

□ 软件体系结构风格

- 描述某一特定应用领域中系统组织方式的惯用模式。

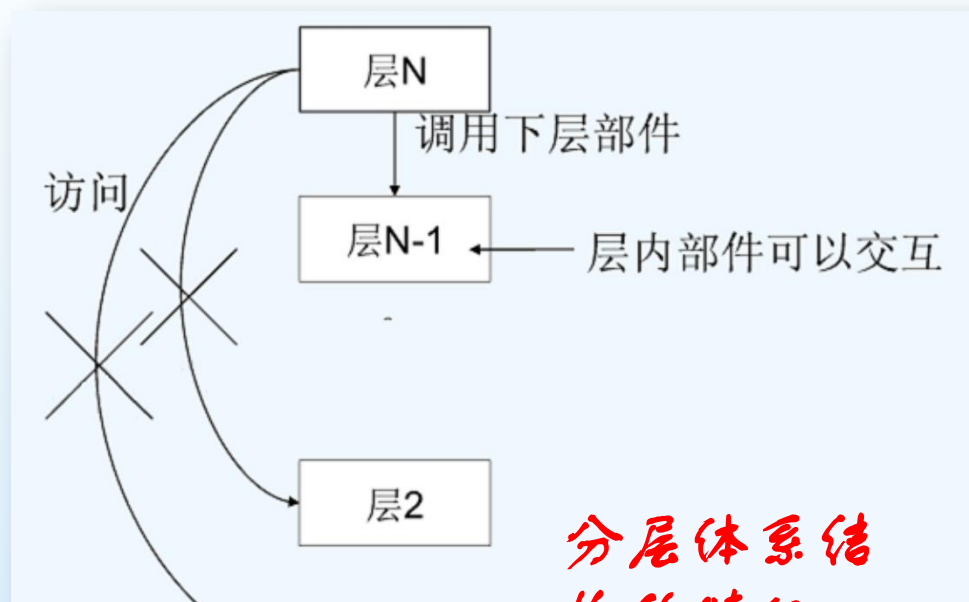
从已有的成功的软件系统的组织结构中抽取出来的

二、软件体系结构风格（模式）

2.1 分层体系结构风格

将系统分成多个水平层次，每个层次只与相邻的层次交互。分层的方法支持系统的增量式开发。这个体系结构还是可改变的和可移植的。如果一个层的接口被保留下来，这个层就能被另外的一个对等层替换。

名字	分层体系结构
描述	将系统组织成分层结构，每一层中包含一组相关的功能。每一层提供服务给毗邻的上一层，因此最底层是有可能被整个系统所使用的核心服务。
使用时机	在已有系统的基础上构建新的设施时使用；当开发团队由多个分散的小团队组成，且每个小团队负责一层的功能时使用；或者当系统存在多层信息安全性需求时使用。



- **关注点分离：**一个层中的构件只会处理本层的逻辑。
- **层间隔离：**不能进行跨层调用，仅使用下层提供的接口。

二、软件体系结构风格（模式）

2、常用的体系结构风格

2.1 分层体系结构风格

高内聚 ←

1.根据单一职责原则的指导思想，应该将业务逻辑与基础设施分离，通过层对多元职责进行分解，每层拥有独立职责，对关注点进行分离，以此应对不同速率的需求变化，使得这些变化可以独立进行。

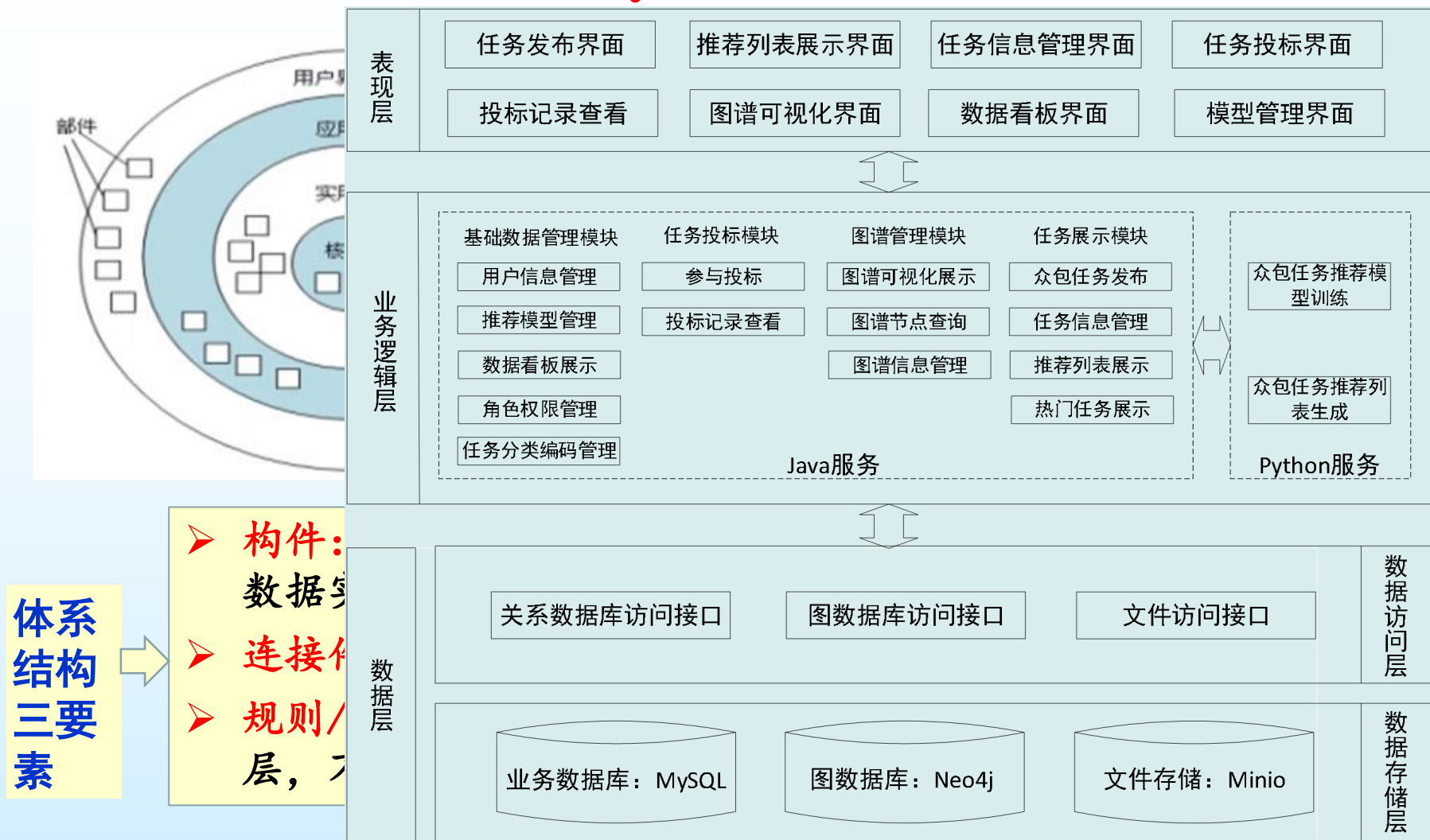
2.根据依赖倒置原则的指导思想，可以通过接口保持分层架构的稳定性，降低不同层级的耦合度，即使某层接口发生变化，也只影响相邻的上层，维护量小且错误可控，面向接口的编程有利于提升可扩展性与代码复用能力。

3.信息隐藏的原则，可以通过层间隔离降低层与层之间的依赖，每个层次对其他层的功能知道的越少越好。不同类型开发人员专注于擅长的开发领域，只关注整个结构中的某一层，这有利于降低开发成本与难度。

↓
低耦合

二、软件体系结构风格（模式）

2.1 分层体系结构（Layered Architecture）



二、软件体系结构风格（模式）

2.1 分层体系结构（Layered Architecture）

□ 分层体系结构的优点：

- 开发人员可以只关注整个结构中的某一层，降低开发与复杂度；
- 层与层之间通过服务接口进行交互，只要接口不变，可以很容易用新的实现替换原有层次的实现，提升了可扩展性与可维护性；
- 某层接口发生变化，只影响相邻上层，维护量小且错误可控，降低了层与层之间的依赖。
- 只要接口不变，层就可以重用。

□ 分层体系结构的缺点：

- 从性能上考虑，要将逻辑上高层次的功能与相对低层次的实现结合起来，每层要处理和解释，效率降低，可通过缓存机制来改善；
- 如果发生根本性需求改变，可能产生跨越多层的级联修改，尤其体现在自上而下的方向，可通过依赖倒置原则改善。

二、软件体系结构风格（模式）

2.1 分层体系结构（Layered Architecture）

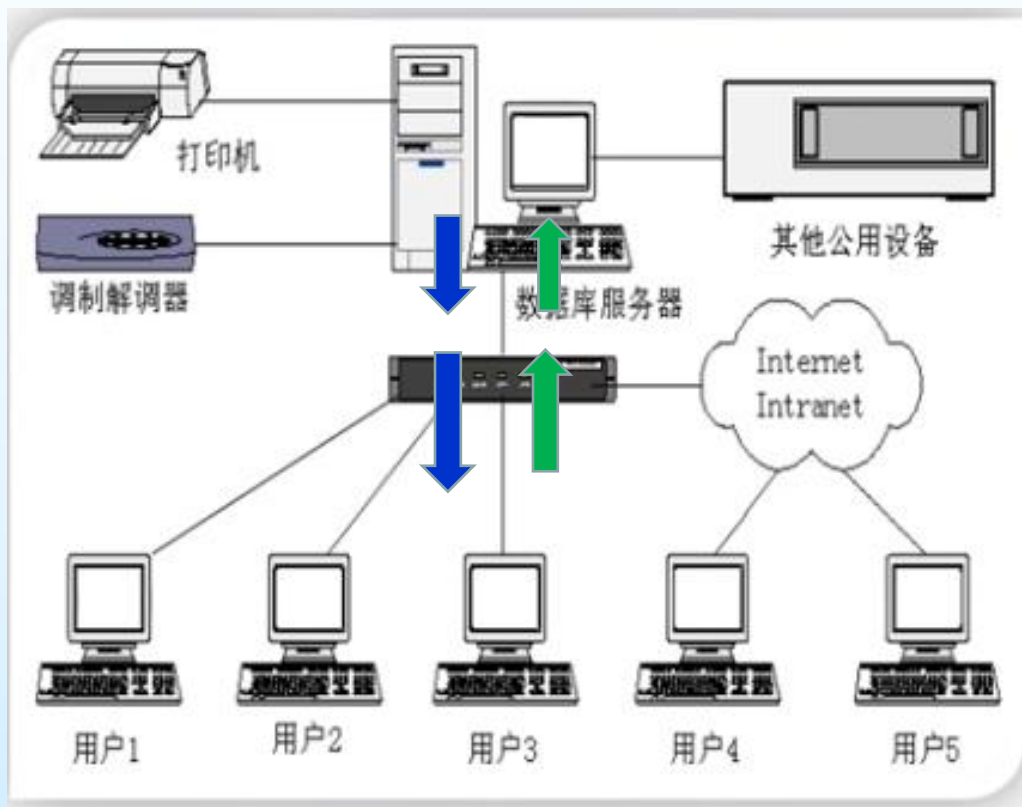
- **适用场景：**适用于需要处理大量数据和复杂业务逻辑的应用程序。
- **企业级应用程序：**企业级应用程序通常需要处理大量数据和复杂业务逻辑，例如ERP、CRM等应用程序。使用分层体系结构可以将系统分为多个水平层次，从而实现分离关注点和松耦合的效果，提高系统的可维护性和可重用性。
- **大型Web应用程序：**大型Web应用程序通常需要处理大量并发请求和复杂业务逻辑，例如电子商务系统、社交媒体平台等。使用分层体系结构可以将系统分为多个水平层次，从而实现分离关注点和松耦合的效果，提高系统的可伸缩性和可维护性。
- **需要满足特定的安全或可靠性要求：**分层体系结构可以将不同的安全或可靠性要求分配到不同的层次中，从而实现安全和可靠性的控制和管理。

二、软件体系结构风格（模式）

2.2 客户端-服务器体系结构（Client-Server Architecture）

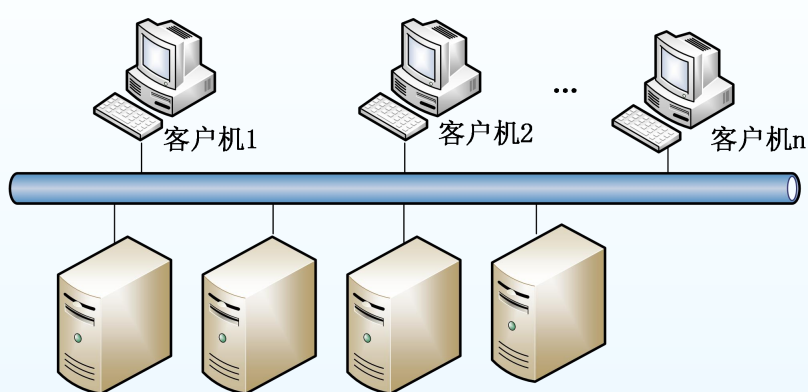
将系统分成客户端和服务端两部分，客户端向服务器发起请求，服务器接收请求并返回响应，客户端进行响应处理。

- Client/Server，简称C/S，20世纪90年代逐渐成熟。
- 目的：基于资源不对等，实现共享。
- 组成部分：服务器、客户应用程序、网络。



二、软件体系结构风格（模式）

2.2 客户端-服务器体系结构（Client-Server Architecture）



文件服务器 Web服务器 Web服务器 数据库服务器

体系结构三要素

- **构件：**客户端程序、数据库服务器
- **连接件：**网络连接
- **规则/配置：**服务器在后台、客户端程序在前台

优点

- 1、强大的数据操作和事务处理能力。
- 2、分布式任务。
- 3、系统具有较强的适应性、灵活性和扩展性。

缺点

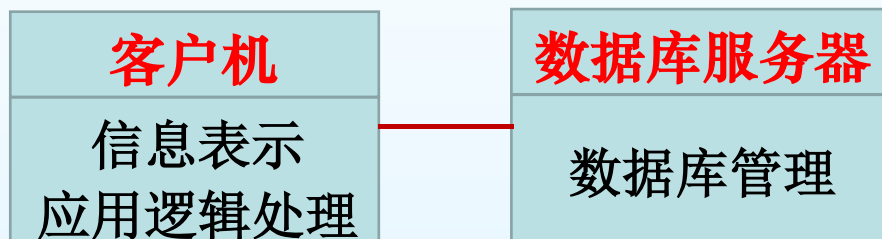
- 1、客户端程序设计复杂、开发成本较高。
- 2、软件移植、软件维护和升级困难。

二、软件体系结构风格（模式）

2.2 客户端-服务器体系结构（Client-Server Architecture）

客户机/服务器结构已被广泛应用基于数据库的信息服务领域，并演变除了多层客户机/服务器结构。

（1）两层结构：将信息表示与应用逻辑处理都放在客户机上，服务器只需要管理数据库事务。这时的客户机成为“胖客户机”。

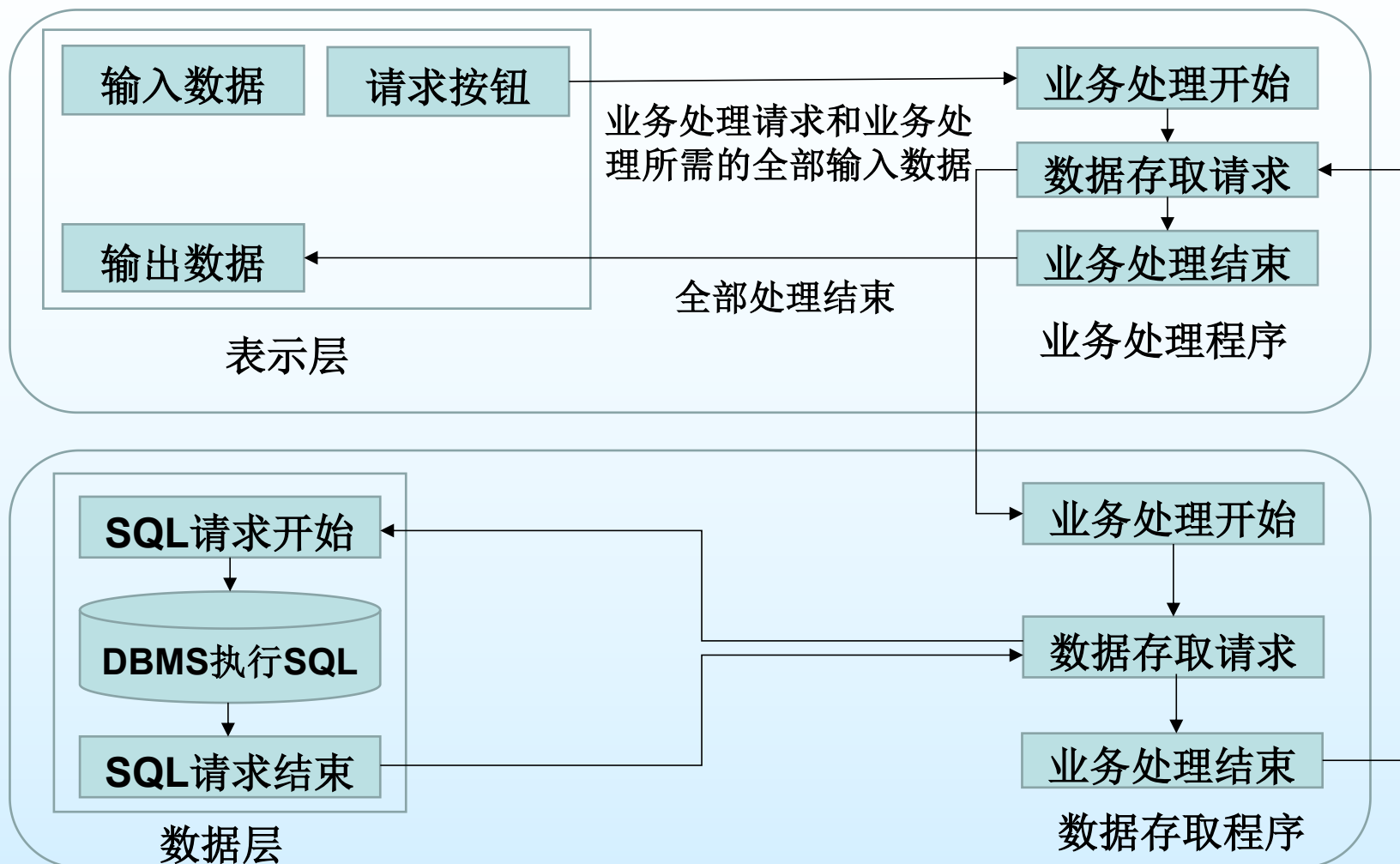


□ **优点：**管理简单、容易实现，能有效保证系统的性能。

□ **缺点：**客户机越多，管理和维护上存在的困难越大。

二、软件体系结构风格（模式）

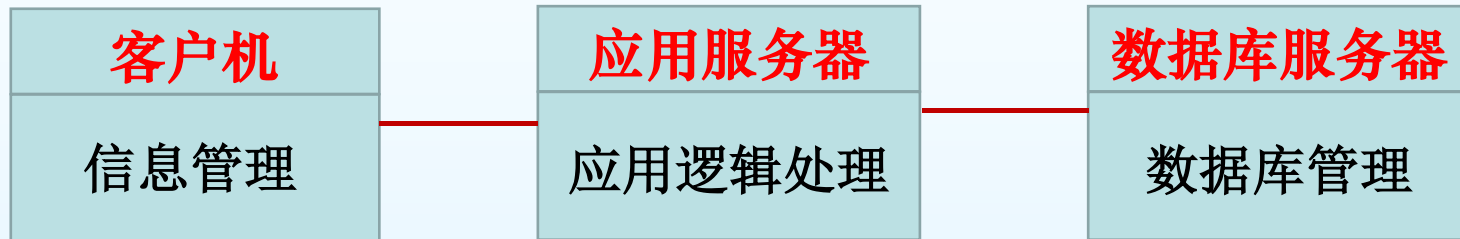
2.2 客户端-服务器体系结构（Client-Server Architecture）



二、软件体系结构风格（模式）

2.2 客户端-服务器体系结构（Client-Server Architecture）

(2) 三层结构：将两层结构的客户机上容易发生变化的应用逻辑部分提取出来，并放到一个专门的“应用服务器”上，使客户机变“瘦”。

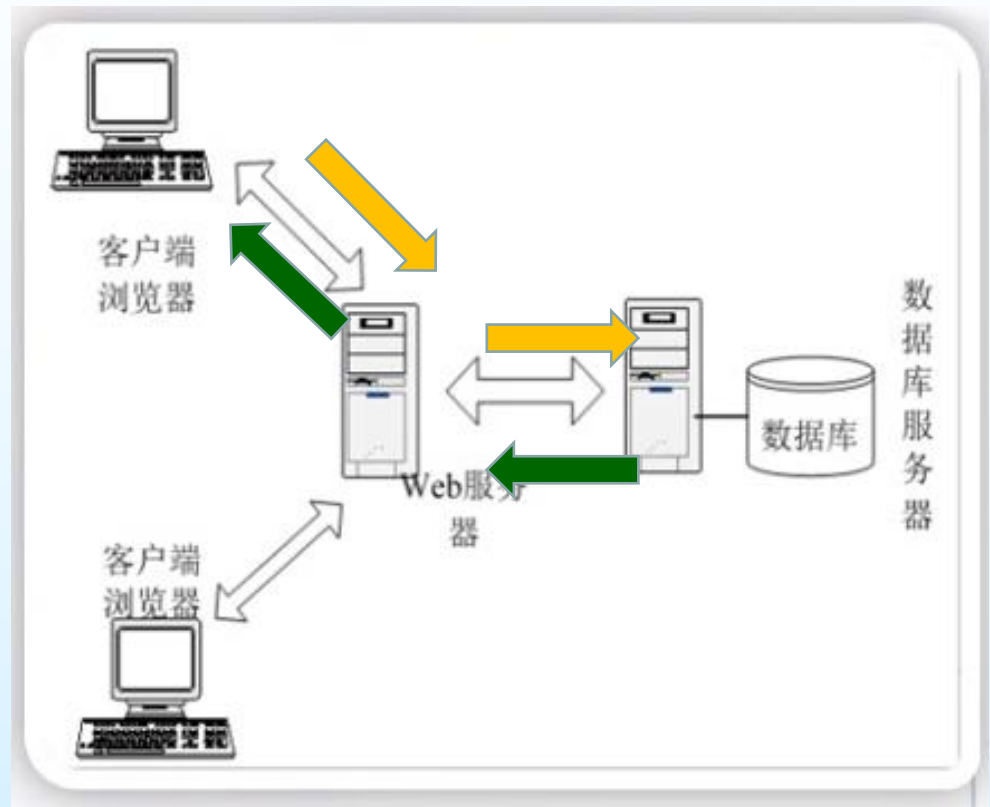


□ 应用服务器：作用是使系统中最易发生需求改变的应用逻辑处理被集中在一起，因此给系统维护带来了很大的便利。当用户应用规则发生变化时，需改变的不是数目庞大的客户端，而是一台或少数几台应用服务器。

二、软件体系结构风格（模式）

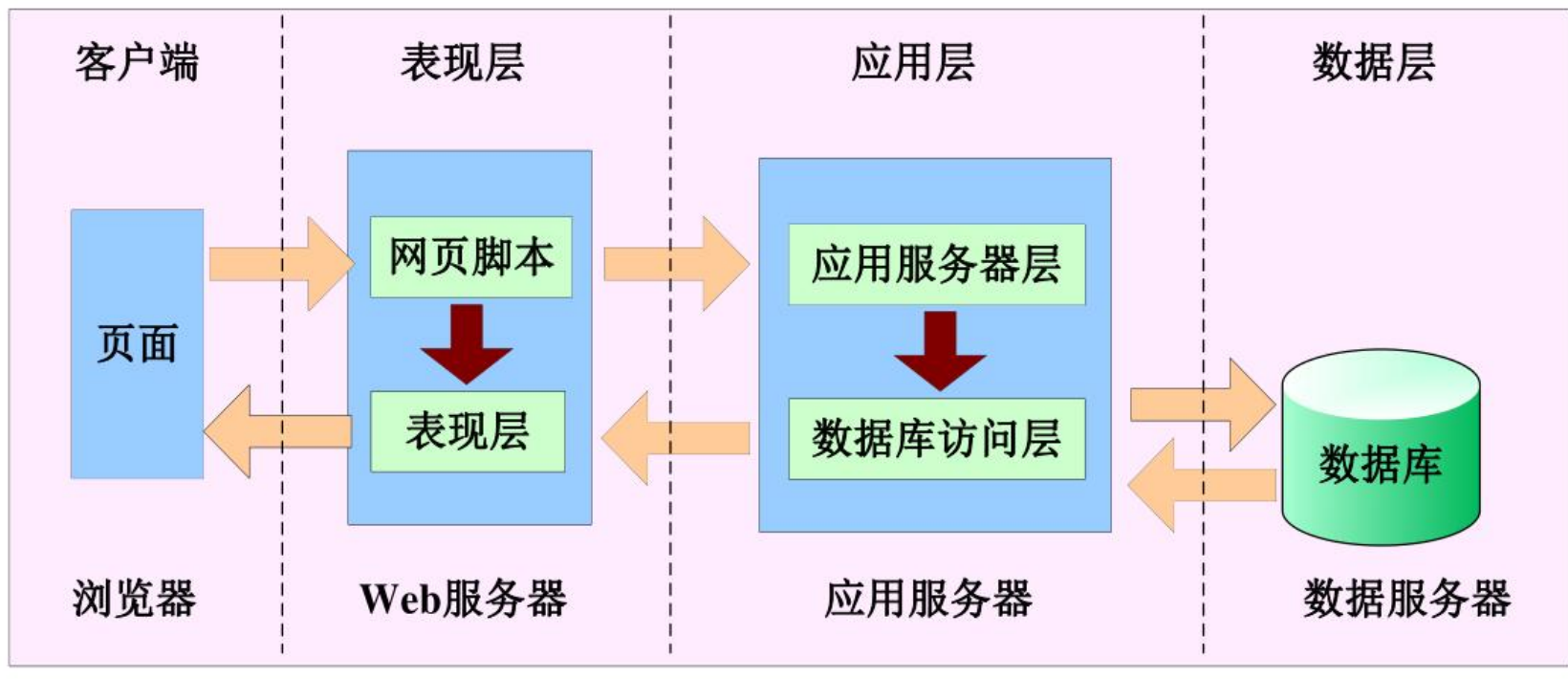
2.2 浏览器-服务器体系结构（Browser-Server Architecture）

- Browser/Server，简称B/S，利用不断成熟的浏览器技术。
- 目的：简化客户端，便于升级。
- 组成部分：数据库服务器、Web服务器、客户端浏览器、网络。



二、软件体系结构风格（模式）

2.2 浏览器-服务器体系结构（Browser-Server Architecture）



客户端无任何业务逻辑，用户在使用系统时，仅仅通过浏览器可运行全部模块。

二、软件体系结构风格（模式）

2.2 浏览器-服务器体系结构（Browser-Server Architecture）

体系结构
三要素



- 构件：客户浏览器、Web服务器、数据库服务器
- 连接件：网络连接（Internet/Intranet）
- 规则/配置：浏览器+Web服务器+数据库服务器分层

□ 优点

- 系统安装、修改和维护在服务器端解决，便于运行时自动升级
- 提供了异种机、异种网、异种应用服务的联机、联网、统一服务的开放性基础。

□ 缺点

- 系统安全性难以控制
- 数据动态交互性不强，数据查询等响应速度低。

二、软件体系结构风格（模式）

2.2 客户端-服务器体系结构（Client-Server Architecture）

□ **适用场景：**适用于需要处理分布式数据和分布式计算的应用程序。

- **处理海量数据：**客户端-服务器体系结构可以将数据处理逻辑和计算逻辑分配到不同的层次中，从而实现分布式计算和分布式数据管理。这种分布式的架构可以提高系统的处理速度和可伸缩性。
- **处理分布式事务：**客户端-服务器体系结构可以将事务处理逻辑分配到不同的层次中，从而实现分布式事务处理和管理。这种分布式的架构可以提高系统的可靠性和可伸缩性。
- **处理分布式网络：**客户端-服务器体系结构可以将网络处理逻辑分配到不同的层次中，从而实现分布式网络管理和控制。这种分布式的架构可以提高系统的可靠性和可伸缩性。

二、软件体系结构风格（模式）

2.3 MVC（模型-视图-控制器）体系结构

□ 问题：

- 用户界面需要频繁的修改，它是“不稳定”的。
- 业务逻辑/数据 与 用户界面之间应尽量少的避免直接通信。

思考：如何让Web应用程序的用户界面与业务逻辑功能实现模块化，以便使程序开发人员可以轻松地单独修改各个部分而不影响其他部分？

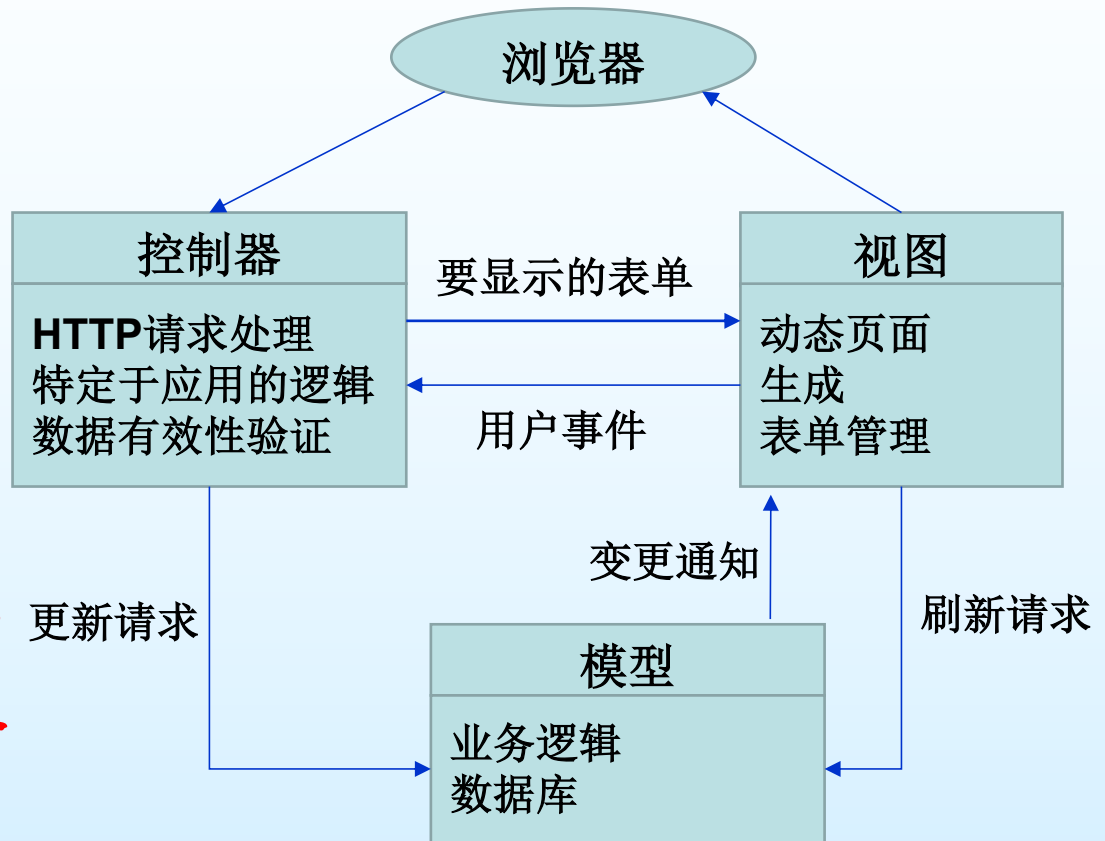
MVC在传统的B/S体系结构的基础上加入了一个新的元素：控制器，由控制器来决定视图与模型之间的依赖关系。

二、软件体系结构风格（模式）

2.3 MVC（模型-视图-控制器）体系结构

MVC模式分离了系统的组成元素，允许他们独立地变更。例如增加一个新的视图或改变一个已有的视图，这些操作都可以在不改变模型底层数据的情况下完成。

保持功能内核独立于用户接口，适用于交互式软件开发



二、软件体系结构风格（模式）

2.3 MVC（模型-视图-控制器）体系结构

- **模型（Model, M）**：用于管理应用系统的行为和数据，并响应为获取其状态信息（通常来自视图）而发出的请求，还会响应更改状态的指令（通常来自控制器）；——对应于传统 B/S 中的业务逻辑和数据。
- **视图（View, V）**：用于管理数据的显示；——对应于传统 B/S 中的用户界面。
- **控制器（Controller, C）**：用于解释用户的鼠标和键盘输入，以通知模型和视图进行相应的更改。——在传统 B/S 结构中新增的元素。

二、软件体系结构风格（模式）

2.3 MVC（模型-视图-控制器）体系结构

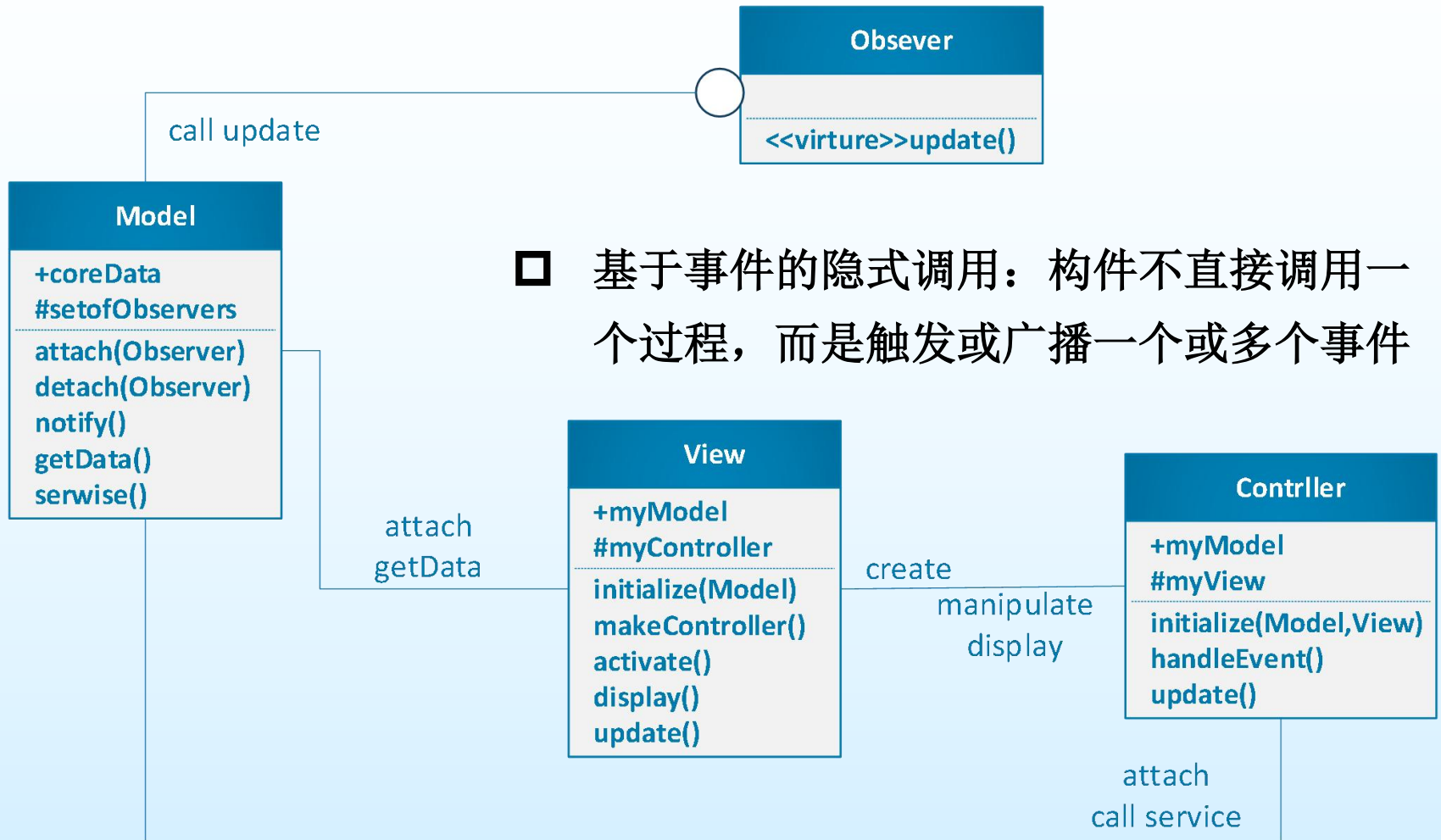
- 视图和控制器组成了用户接口，变更-传播机制保证了模型和用户接口之间的一致性。
- 模型组件包含了应用程序的功能内核，封装了相应的数据并输出执行特定应用程序处理的过程。
- 控制器代表用户调用这些过程。

变更-传播机制

- 维护了一个模型中相依组件的注册表。
- 所有视图还有一些控制器在这个表中登记了对变更通知的需求。
- 模型状态的改变将触发变更-传播机制。每个在表中登记的视图和控制器都会收到变更通知。
- 变更-传播机制是模型与视图、控制器之间唯一连接。

二、软件体系结构风格（模式）

2.3 MVC（模型-视图-控制器）体系结构



- 基于事件隐式调用：构件不直接调用一个过程，而是触发或广播一个或多个事件

二、软件体系结构风格（模式）

2.3 MVC（模型-视图-控制器）体系结构

□场景I——用户输入导致模型变化，并触发变更传播机制

- 控制器接受到事件，解释事件并启动模型的服务过程；
- 模型执行相应的过程，并导致内部状态的变化；
- 模型调用其更新过程，向所有登记请求了变更-传播机制的视图和控制器发出通知；
- 每个视图从模型中读取数据并重新显示；
- 每个控制器修改自己的行为，比如禁用某个功能；
- 最初的控制器恢复控制并从事件处理过程返回。

二、软件体系结构风格（模式）

2.3 MVC（模型-视图-控制器）体系结构

□ 场景II——初始化过程

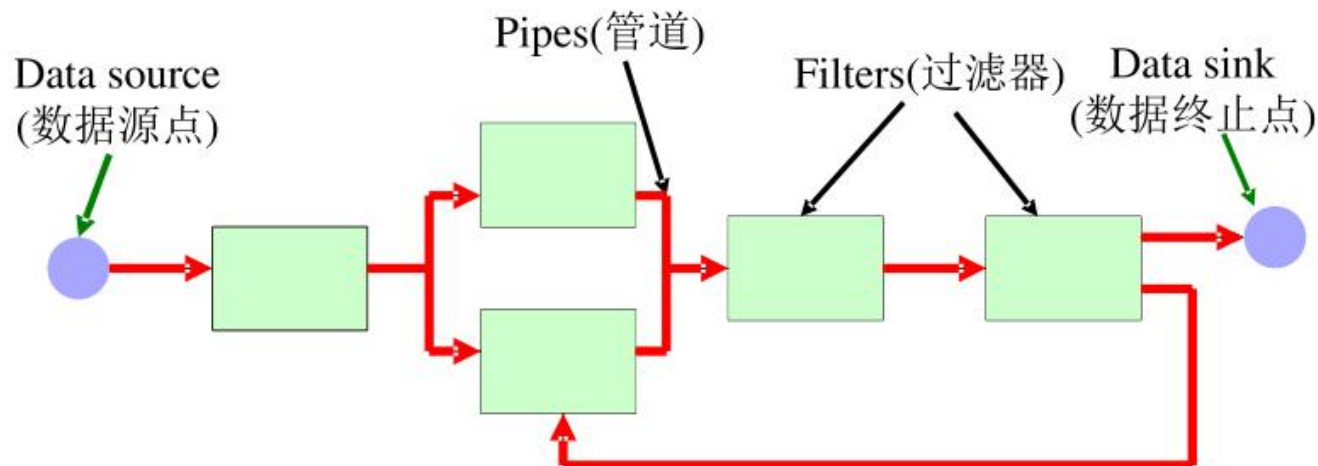
- 创建模型实例，并初始化其数据；
- 创建视图对象，并对模型的引用作为其初始化参数之一；
- 视图通过调用附属过程支持变更-传播机制；
- 视图创建控制器，此时将模型和视图的引用作为参数传递给控制器初始化过程；
- 控制器通过调用附属过程来支持变更-传播机制；
- 初始化完成，应用程序开始处理事件。

□ **适用场景：** 应用程序需要分离业务逻辑和用户界面，或需要支持多种用户界面，优先选择MVC体系结构可能更为合适。例如，可以使用相同的模型和控制器来支持Web应用程序、移动应用程序和桌面应用程序等不同的用户界面。

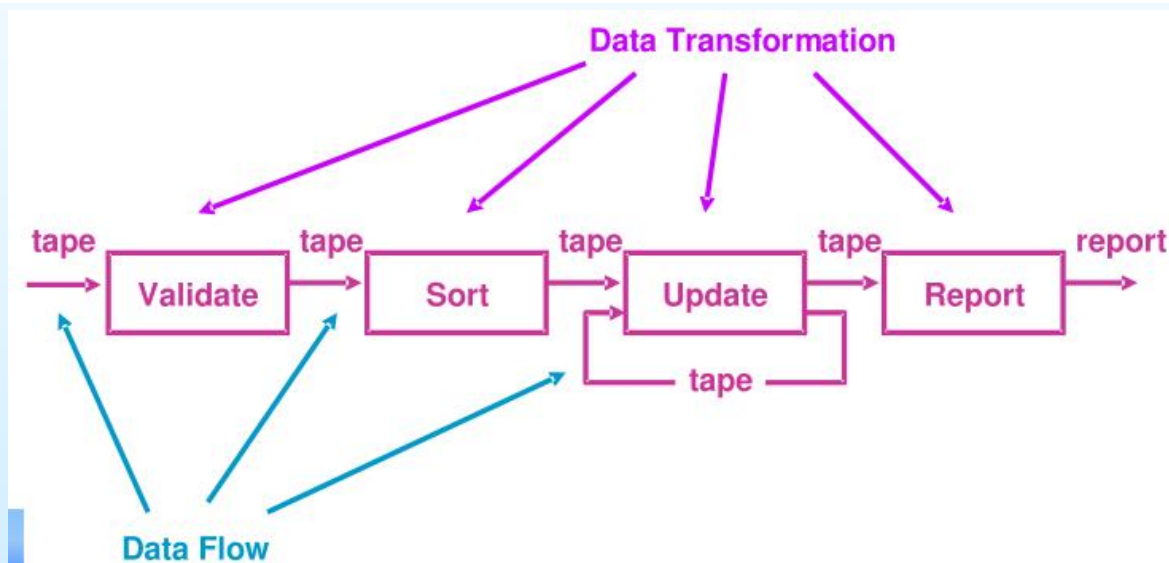
二、软件体系结构风格（模式）

2.4 数据流体系结构（Data-flow Architecture）

(1) 管道
过滤器风格



(2) 批处
理风格



二、软件体系结构风格（模式）

2.4 数据流体系结构（Data-flow Architecture）

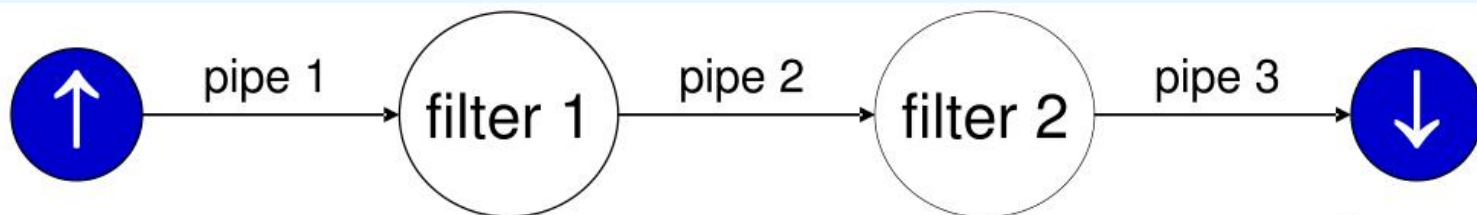
□ **语境：**数据源源不断的产生，系统需要对这些数据进行若干处理（分析、计算、转换等）。

□ **解决方案：**

--把系统分解为几个连续的处理步骤，这些步骤之间通过数据流连接，一个步骤的输出是另一个步骤的输入；

--每个处理步骤由一个过滤器构件（Filter）实现；

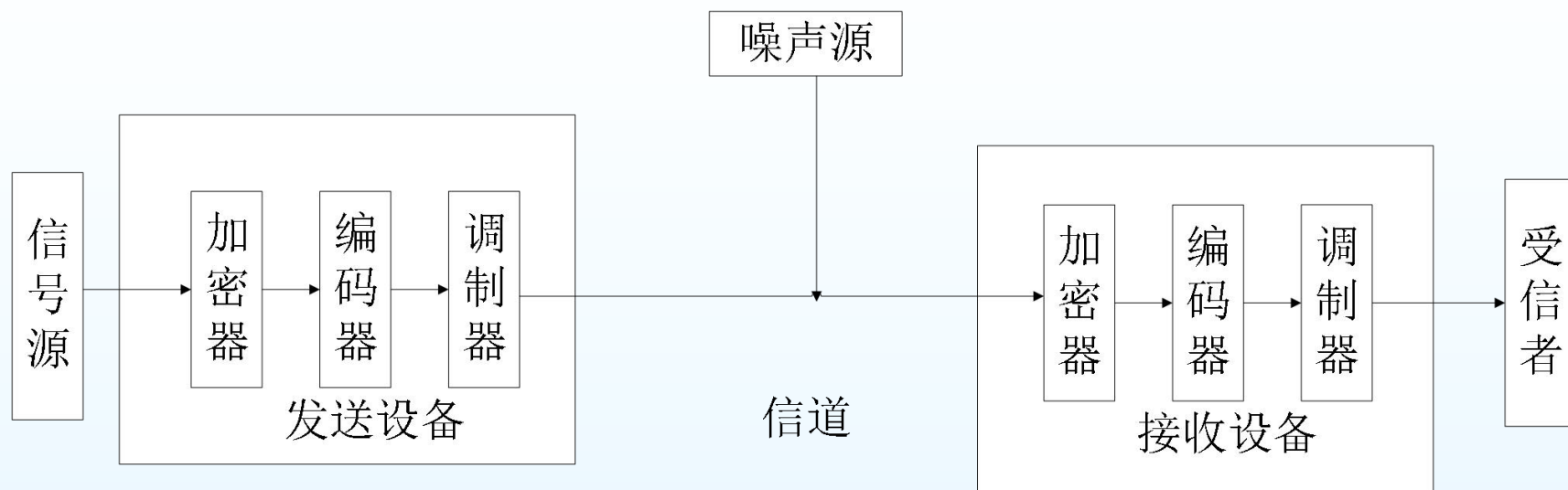
--处理步骤之间的数据传输由管道（Pipe）负责。



□ **典型应用：**编译器、Unix管道、图像处理、信号处理、网络监控与管理等。

二、软件体系结构风格（模式）

2.4 数据流体系结构（Data-flow Architecture）



数字通信系统模型

管道过滤器风格的优点

- 设计人员将整个系统的输入输出行为理解为单个过滤器行为的叠加与组合。
- 任何两个过滤器，只要它们之间传递的数据遵守共同的规约就可以相连接。
- 整个系统易于维护和升级。

二、软件体系结构风格（模式）

2.5 数据共享体系结构（Data-Sharing Architecture）

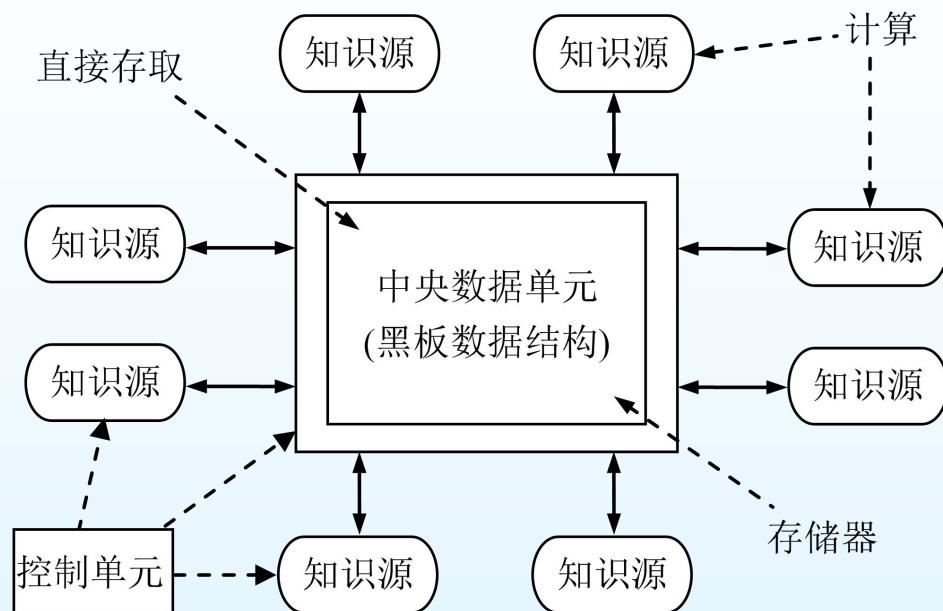
数据共享体系结构描述的是一组交互组件如何共享数据。

名字	容器
描述	<u>系统的所有数据在一个中央容器中管理，该中央容器可以被所有系统组件访问</u> 。组件间不是直接进行交互，它们只通过容器进行交互
使用时机	当一个系统中所生成的大量信息需要持久保存时，可以使用该模式。也可以在数据驱动系统中使用该模式，每当在容器中收入数据时将触发一个动作或工具
优点	组件是独立的，他们无需知道其他组件的存在。一个组件的变更可以传播到所有的组件，所有的数据可以得到一致的管理（例如在同一时间进行备份），因为它们存放在同一地方
缺点	容器是一个单个失效点，因而容器中的问题会影响整个系统。在组织所有通过容器进行通信时会比较低效。将容器分布到多个计算机上会很困难

二、软件体系结构风格（模式）

2.5 数据共享体系结构（Data-Sharing Architecture）

数据共享风格有两种不同类型的软件元素：一种是**中央数据单元**，也称为资源库，用于表示系统的当前状态，另一种是**相互依赖的构件组**。中央数据单元和构件之间可以进行信息交换。

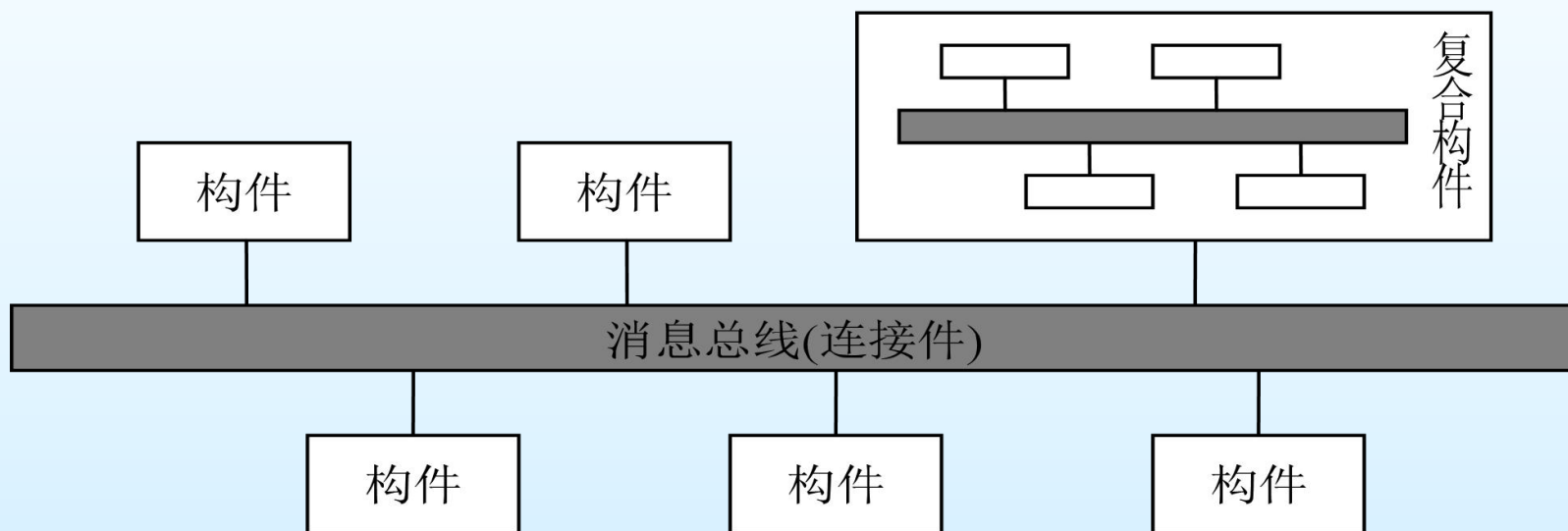


- **适用场景：**数据共享体系结构通常适用于数据密集型的应用程序，例如金融系统、医疗系统、大型企业信息系统等。在这些应用程序中，数据通常是系统的核心资源，不同的业务组件需要访问和共享数据来完成其功能。使用数据共享体系结构可以将数据抽象出来，并提供一种标准的数据访问接口，使得各个业务组件可以通过这个接口访问数据，从而达到数据共享和数据复用的目的。

二、软件体系结构风格（模式）

2.6 总线体系结构（Bus Architecture）

层次消息总线的理论基础是消息驱动的编程方法和计算机硬件总线概念，**消息总线**负责在构件之间传递消息，根据构件—消息响应登记表把消息分派给对其感兴趣的**构件**，并返回处理结果。杂构件分解为粒度更细的子构件，通过局部消息总线进行连接，从而形成复合构件。



二、软件体系结构风格（模式）

2.6 总线体系结构（Bus Architecture）

□ 优点

- **简单：**总线体系结构使用共享的总线作为通信介质，避免了复杂的点对点通信协议，使系统结构更加简单明了。
- **灵活：**由于总线的存在，各组件可以方便地添加或移除，系统结构更加灵活。
- **可扩展：**总线体系结构可以很容易地进行扩展，添加更多的组件和设备。

□ 缺点

- **总线瓶颈：**由于所有的组件都通过总线进行通信，当总线带宽有限时，系统的性能可能会受到限制。
- **单点故障：**当总线发生故障时，整个系统都可能会受到影响。
- **实时性限制：**在实时系统中，总线的使用可能会导致通信延迟，影响系统的实时性能。

二、软件体系结构风格（模式）

2.6 总线体系结构（Bus Architecture）

□ 优点

- 简单：总线体系结构使用共享的总线作为通信介质，避免了复杂的点对点通信协议，使系统结构更加简单明了。
- 灵活：由于总线的存在，各组件可以方便地添加或移除，系统结构更加灵活。
- 可扩展：总线体系结构可以很容易地进行扩展，添加更多的组件和设备。

□ 缺点

- 总线瓶颈。由于所有的组件都通过总线进行通信，当总线带宽有限时，系统

□ 适用场景：总线体系结构通常适用于小型系统和嵌入式系统等简单的应用场景，例如传感器网络、控制系统、自动化系统、家庭娱乐系统等。在这些场景中，系统的规模相对较小，通信需求简单，使用总线体系结构可以提高系统的可扩展性和灵活性，同时降低系统的复杂性和成本。

二、软件体系结构风格（模式）

3、体系结构的设计决策

体系结构模式是一种经过实践验证、可以在系统中复用的软件体系结构的组织方式。

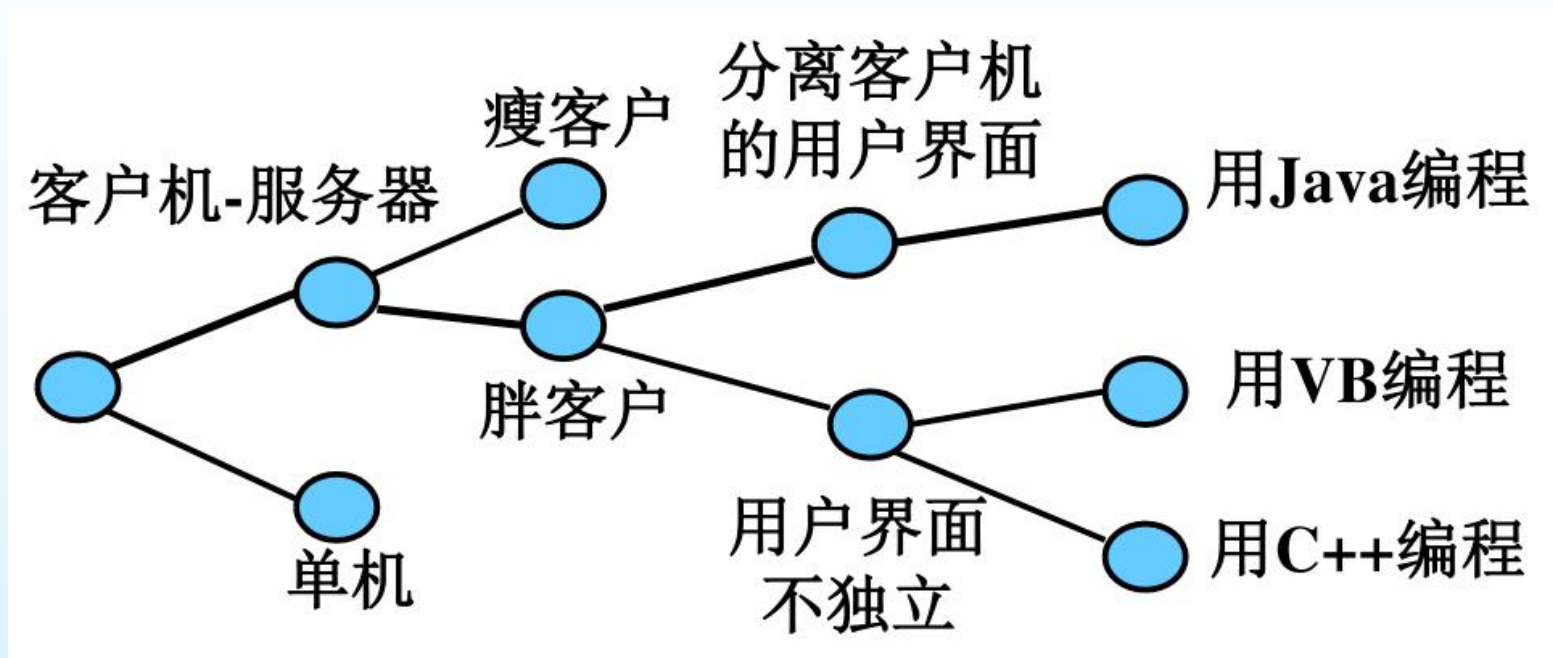
体系结构设计是一个创造性的过程，在这个过程中你会设计一个满足系统功能性和非功能性需求的系统组织结构。并不存在公式化的体系结构设计过程，具体体系结构设计取决于所开发的系统的类型、系统架构师的经验、背景，以及系统的特定需求。系统架构师应考虑的问题：

- 是否存在一个通用的体系结构可用作设计的模板？
- 系统如何分布到各个硬件或处理器上？
- 用什么样的策略来控制系统中构件的运行？

二、软件体系结构风格（模式）

3、体系结构的设计决策

解决“how to do”，就需要不断的做出各种“设计决策”，在各类需求之间进行“折中”，使得最终设计性能达到最优。



二、软件体系结构风格（模式）

3、体系结构的设计决策

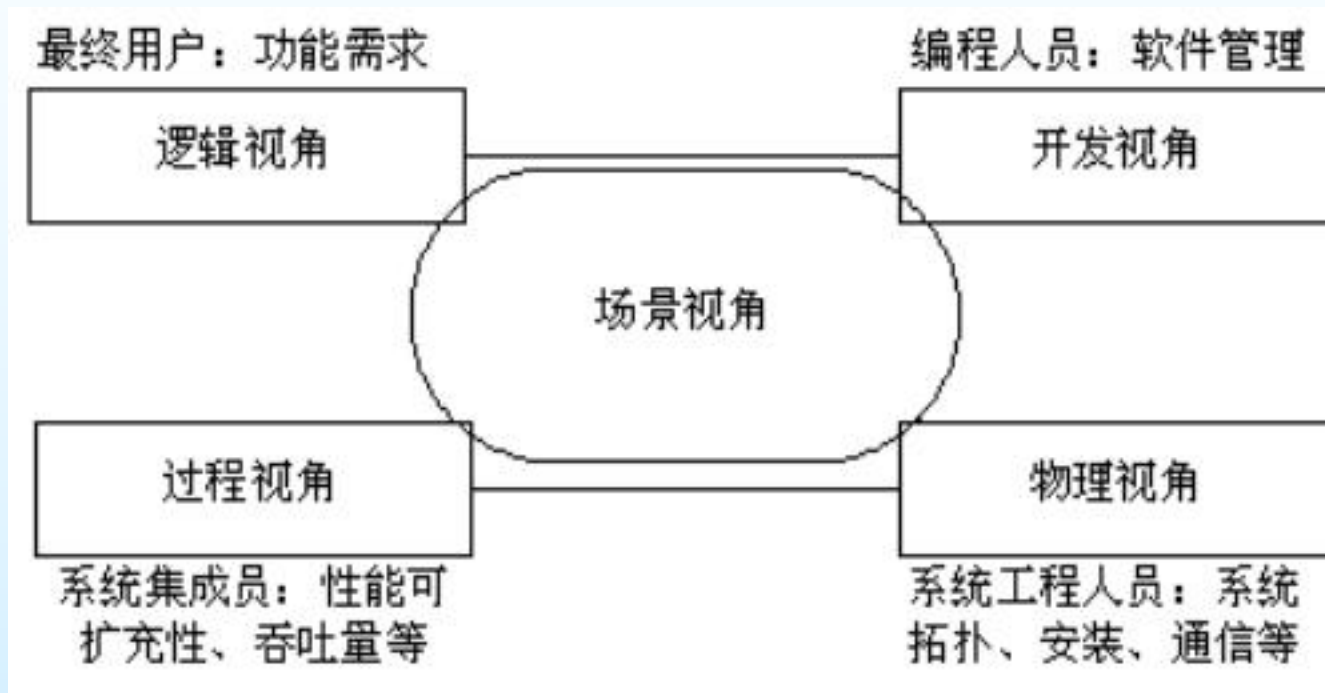
体系结构风格的选择应该根据系统的非功能性需求来决定：

- **性能**。如果性能是一个关键性因素，那么体系结构应当将关键性操作局部化到少量构件中，尽量让这些构件部署在同一台计算机中而不要分布到网络上。同时意味着适用大粒度的构件比小粒度的构件更节省通信开销。
- **信息安全**。如果信息安全是一个关键性需求，那么应该使用一种层次化体系结构，把最关键的资产放在最内层。
- **可用性**。如果可用性是一个关键性指标，那么体系结构设计应包含冗余构件以便在不停止系统的情况下可以更换或替换构件。
- **可维护性**。如果可维护性是一个关键性指标，那么体系结构设计应使用变更的细粒度、自包含的构件。应当将数据的生产者与消费者分离。

三、软件体系结构描述

■ 软件体系结构的“4+1”视图模型

Rational公司的Kruchten在1995年提出“4+1”视图模型：
用逻辑视图、进程视图、物理视图、开发视图和场景视图来描述软件体系结构。



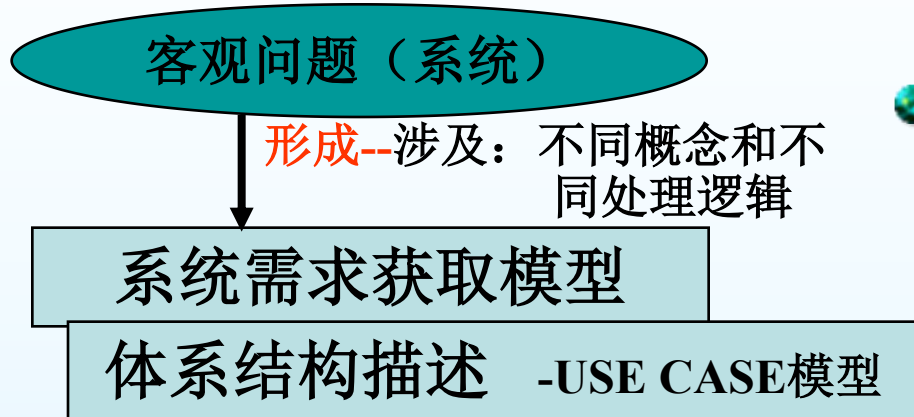
每一个视图只关心系统的一个侧面，5个视图结合在一起才能反映系统的软件体系结构的全部内容。

三、软件体系结构描述

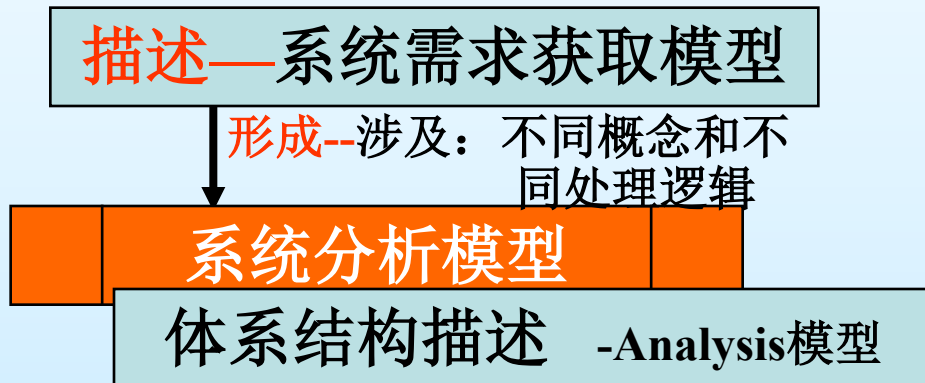
1、逻辑视图

◆ 面向最终用户

① 需求获取的目标是：



② 需求分析的目标是：



● 逻辑视图主要支持功能需求——系统应向用户提供什么样的服务。

在逻辑视图中，系统分解成一系列功能抽象，这些抽象主要来自问题领域。这种分解不但可以用来进行功能分析，而且可用作标识在整个系统的各个不同部分的通用机制和设计元素。

三、软件体系结构描述

1、逻辑视图

◆ 面向最终用户

逻辑视图描述了系统的功能和数据处理的逻辑组成部分，包括系统的功能需求、功能分解、对象模型、类图、状态图等。在面向对象技术中，通过抽象、封装和继承，可以用对象模型来代表逻辑视图，用类图来描述逻辑视图。



- 逻辑视图主要通过[类图](#)、[顺序图](#)、[活动图](#)等来表示软件系统中的逻辑组成和交互关系，以便于理解和设计

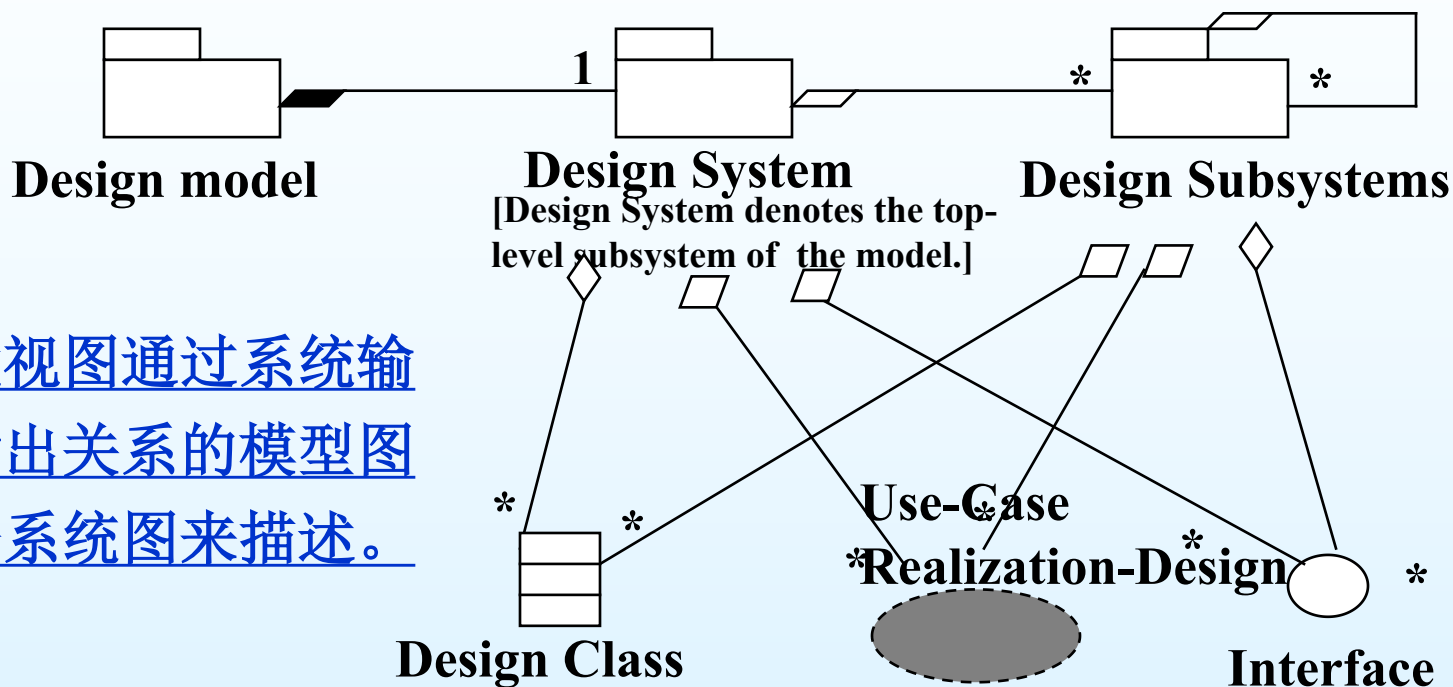
三、软件体系结构描述

2、开发视图

◆ 面向系统开发人员

开发视图也称模块视图，主要侧重于软件模块的组织和管理。

- 开发视图通过系统输入输出关系的模型图和子系统图来描述。



- ✓ 表达子系统的分解，它们的接口以及相互间的依赖。这一分解对体系结构非常有意义，因为子系统和它们的接口构成了系统的基本结构。

三、软件体系结构描述

2、开发视图

◆ 面向系统开发人员

在开发视图中最好采用4-6层子系统，而且每个子系统仅仅能与同层或更低层的子系统通讯，避免各模块间复杂的依赖关系。



开发视图主要通过包图、类图、部署图等来表示软件系统的开发过程和开发者所需要的信息，以便于管理和组织。

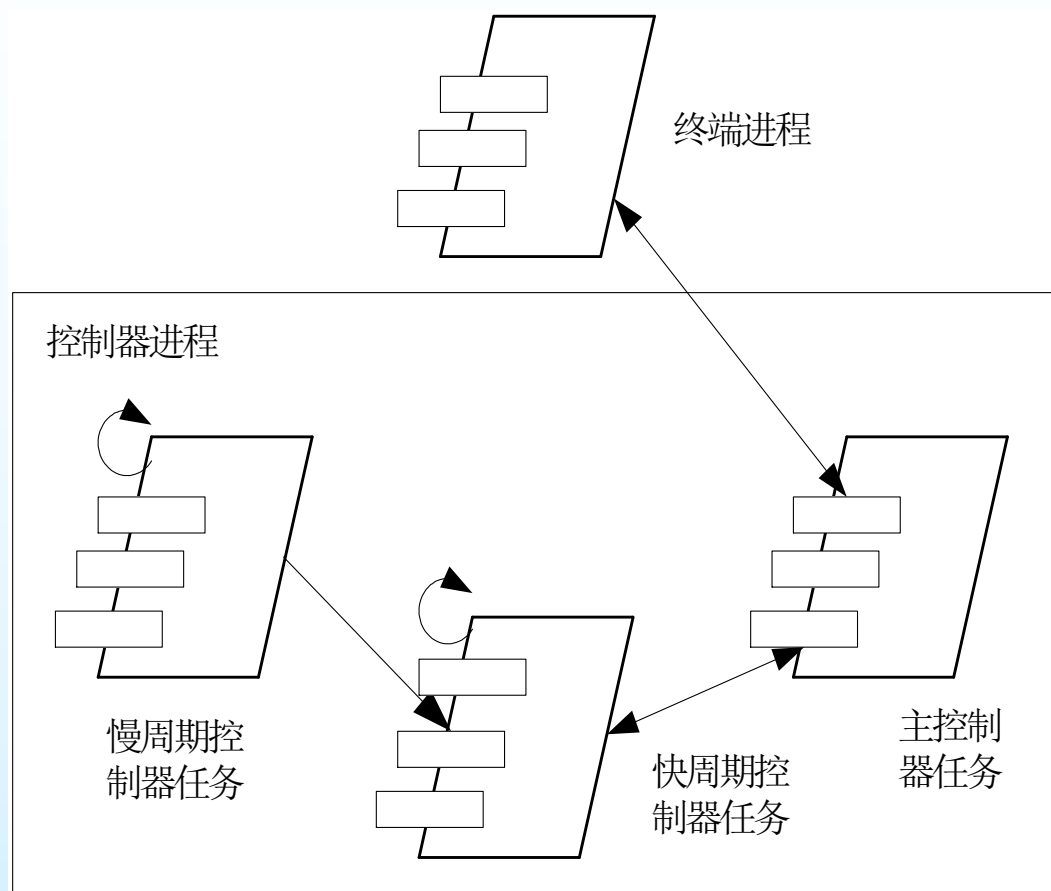
三、软件体系结构描述

3、进程视图

进程视图侧重于系统的运行特性，主要关注一些非功能性的需求。

进程视图描述了软件系统的并发性、同步性和互操作性，包括系统的进程、线程、同步、并发、通信等。它也定义逻辑视图中各个类的操作具体是在哪一个线程中被执行的。

◆ 面向系统集成人员

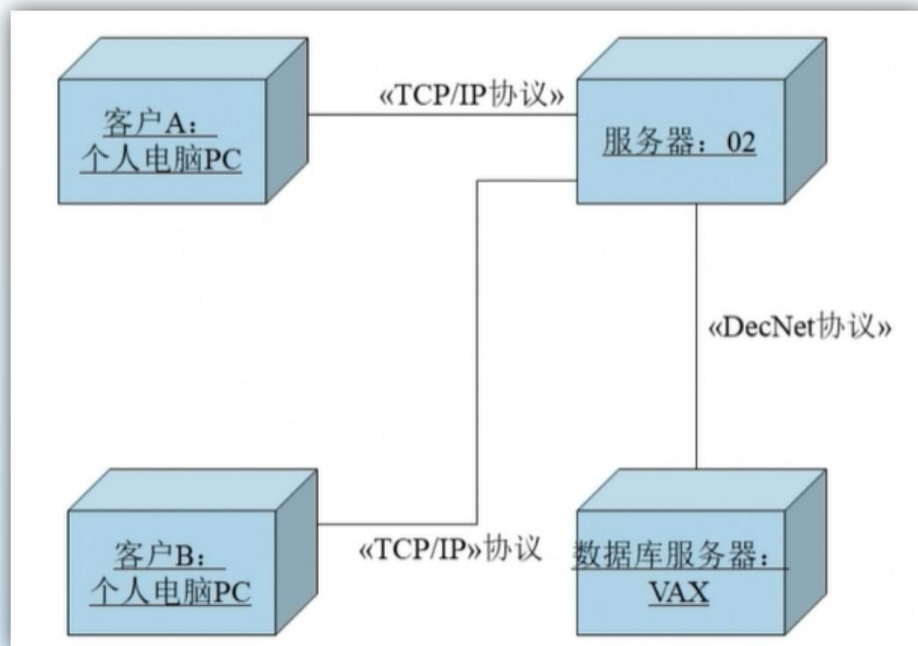


三、软件体系结构描述

4、部署视图

◆ 面向系统部署人员

部署视图是一个对象模型，描述了系统的物理分布，即怎样把功能分布于各个结点（nodes）。



- ◆ 部署视图主要考虑如何把软件映射到硬件上，通常考虑系统性能、规模、可靠性等；
- ◆ 解决系统拓扑结构、安装、通信等问题；

◆ 考虑的是非功能性的系统需求，如系统的可用性、可靠性（容错性）、性能（信息吞吐量）和可扩展性。

三、软件体系结构描述

5、场景视图

场景视图的体系结构：汇总

通过使用一些重要场景，4个视图中的元素可以协调地共同工作。尽管这些场景是一个小集合，但是它们很重要。场景（scenario）是更通用的概念——用例（use case）的实例。从某种意义上讲，场景是最重要的需求的抽象。场景的设计使用对象场景图和对象交互图来表示。

相对于其他的4个视图，这个视图是多余出来的，但是它承担着两个目的：

- 在体系结构设计中，将以此视图为驱动来发现体系结构元素。
- 在体系结构设计结束后，此视图承担验证和描述的角色。它不仅用于书面记录，并且是体系结构原型测试的起始点。

谢谢观看

